

Introdução

Operações CRUD

As quatro operações fundamentais

CREATE (Criar)

 A operação **Create** permite inserir novos dados no sistema. É o ponto de partida do ciclo de vida de qualquer informação. Sem criação, não existem dados para gerir.

No contexto empresarial:

A criação de dados acontece quando:

- Um novo cliente se regista no sistema
- Um colaborador adiciona um produto ao catálogo
- É emitida uma nova fatura
- Um utilizador submete um novo pedido de suporte

Principais características:

- Adiciona novos registos à base de dados
- Requer validação dos dados inseridos (formato, obrigatoriedade, unicidade)
- Pode gerar identificadores únicos automaticamente
- Frequentemente desencadeia outras ações (envio de emails, notificações)

READ (Ler)

❗ A operação **Read** permite consultar e visualizar os dados existentes sem os modificar. É a operação mais frequente em qualquer sistema, representando normalmente 70-90% de todas as interações com a base de dados.

No contexto empresarial:

A leitura de dados ocorre quando:

- Consultamos o histórico de compras de um cliente
- Geramos os relatórios de vendas mensais
- Visualizamos o stock disponível de um produto
- Pesquisamos as faturas por período ou cliente

Principais características:

- Não altera o estado dos dados
- Pode envolver filtros, ordenação e agregação
- Fundamental para tomada de decisões empresariais
- Deve ser otimizada para garantir performance

UPDATE (Atualizar)

 A operação **Update** permite modificar os dados já existentes no sistema, mantendo-os corretos e relevantes ao longo do tempo.

No contexto empresarial:

A atualização de dados acontece quando:

- Um cliente altera a sua morada de entrega
- Atualizamos o preço de um produto
- Modificamos o estado de uma encomenda (pendente → enviada → entregue)
- Corrigimos as informações incorretas ou desatualizadas

Principais características:

- Modifica os registos existentes sem criar novos
- Pode afetar um ou múltiplos registos simultaneamente
- Requer cuidado para não perder informação relevante
- Muitas vezes mantém-se histórico das alterações (auditoria)

DELETE (Eliminar)

 A operação **Delete** remove os dados do sistema. É a operação mais sensível e requer maior cuidado, pois pode resultar em perda permanente de informação.

No contexto empresarial:

A eliminação de dados ocorre quando:

- Um cliente solicita a remoção dos seus dados (RGPD)
- Removemos os produtos descontinuados do catálogo
- Eliminamos os registos duplicados
- Apagamos os dados temporários ou obsoletos

Principais características:

- Remove permanentemente registos da base de dados
- Frequentemente substituída por "soft delete" (marcar como inativo)
- Pode ter implicações legais (retenção de dados, RGPD)
- Deve respeitar a integridade referencial (relações entre tabelas)

Ciclo de vida dos dados

Os dados numa organização seguem um ciclo de vida natural e as operações CRUD mapeiam diretamente cada fase deste ciclo:

1 Nascimento (CREATE)

Os dados entram no sistema pela primeira vez.

Exemplo: Um novo cliente regista-se na plataforma de e-commerce.

2 Utilização ativa (READ e UPDATE)

Os dados são consultados regularmente e atualizados, conforme necessário.

Exemplo: O cliente faz compras, atualiza a morada, consulta encomendas.

3 Manutenção (UPDATE)

Os dados são corrigidos ou enriquecidos.

Exemplo: Adicionamos informações complementares ao perfil do cliente.

4 Fim de vida (DELETE)

Os dados são removidos quando deixam de ser necessários ou por requisitos legais.

Exemplo: O cliente solicita a remoção da conta.

Considerações importantes

- Nem todos os dados percorrem o ciclo completo (alguns nunca são eliminados)
- A frequência de cada operação varia conforme o tipo de dado
- Dados críticos podem ter políticas específicas de retenção
- O ciclo pode incluir fases de arquivo antes da eliminação definitiva

CRUD em SQL (bases de dados relacionais)

Em bases de dados relacionais geridas através de SQL, as operações CRUD correspondem a comandos específicos da linguagem.

Operação CRUD	Comando SQL	Descrição
CREATE	INSERT INTO	Adiciona novos registos a uma tabela
READ	SELECT	Consulta os dados de uma ou mais tabelas
UPDATE	UPDATE	Modifica os registos existentes
DELETE	DELETE FROM	Remove os registos de uma tabela

Exemplo de estrutura básica

```
-- CREATE
INSERT INTO clientes (nome, email, telefone)
VALUES ('João Silva', 'joao@empresa.pt', '912345678');

-- READ
SELECT nome, email FROM clientes WHERE ativo = true;

-- UPDATE
UPDATE clientes SET telefone = '918765432'
WHERE email = 'joao@empresa.pt';

-- DELETE
DELETE FROM clientes WHERE email = 'joao@empresa.pt';
```

Características do SQL

- Estrutura rígida e definida (schema)
- Garante a integridade referencial entre tabelas
- Suporta as transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade)
- Ideal para os dados estruturados e relacionados

CRUD em NoSQL (bases de dados não relacionais)

Nas bases de dados NoSQL, as operações CRUD mantêm o mesmo propósito, mas a sintaxe e abordagem variam, conforme o tipo de base de dados (documentos, chave-valor, colunas, grafos).

Exemplo com MongoDB (base de dados de documentos)

```
// CREATE
db.clientes.insertOne({
  nome: "João Silva",
  email: "joao@empresa.pt",
  telefone: "912345678",
  ativo: true
});

// READ
db.clientes.find({ ativo: true });

// UPDATE
db.clientes.updateOne(
  { email: "joao@empresa.pt" },
  { $set: { telefone: "918765432" } }
);

// DELETE
db.clientes.deleteOne({ email: "joao@empresa.pt" });
```

Principais diferenças do NoSQL

- Estrutura flexível (schema-less ou schema-on-read)
- Otimizado para escalabilidade horizontal
- Diferentes modelos de consistência (eventual consistency)
- Ideal para os dados não estruturados ou semi-estruturados

Boas práticas nas operações CRUD

Independentemente da tecnologia utilizada, existem princípios fundamentais a seguir.

1 Validação de dados

- Verificar o formato e tipo de dados antes de inserir
- Garantir a obrigatoriedade de campos essenciais
- Validar a unicidade (emails, números de identificação)

3 Integridade dos dados

- Respeitar as relações entre tabelas
- Utilizar as transações para operações múltiplas
- Implementar as constraints adequadas

5 Gestão de erros

- Tratar as exceções adequadamente
- Fornecer as mensagens de erro compreensíveis
- Implementar os mecanismos de rollback quando necessário

2 Segurança

- Implementar a autenticação e autorização
- Proteger contra SQL injection e outras vulnerabilidades
- Registrar as ações sensíveis (auditoria)

4 Performance

- Criar os índices em campos frequentemente consultados
- Limitar os resultados de queries grandes (paginação)
- Evitar as operações desnecessárias

CRUD em aplicações empresariais

Na prática, as operações CRUD raramente acontecem isoladamente. Uma ação do utilizador pode desencadear múltiplas operações:

Exemplo: Processar uma encomenda

- **CREATE:** Criar um registo da encomenda
- **READ:** Consultar o stock disponível
- **UPDATE:** Atualizar as quantidades em stock
- **CREATE:** Gerar a fatura
- **UPDATE:** Atualizar o histórico do cliente

Exemplo: Sistema de gestão de recursos humanos

- **CREATE:** Registrar um novo colaborador
- **READ:** Consultar as férias disponíveis
- **UPDATE:** Processar as atualizações salariais
- **DELETE:** Remover os acessos de colaboradores que saíram

ESTUDO DE CASO

Sistema de gestão de produtos para e-commerce

Contexto

A empresa **TechStore Portugal** é uma loja online de equipamentos informáticos que necessita de um sistema para gerir o seu catálogo de produtos. O gestor de produtos, Ricardo, precisa de realizar as operações diárias para manter o catálogo atualizado e responder às necessidades do negócio.

A base de dados possui uma tabela produtos com a seguinte estrutura:

Tabela: produtos

- id (inteiro, chave primária, auto-incremento)
- nome (texto, obrigatório)
- categoria (texto, obrigatório)
- preco (decimal, obrigatório)
- stock (inteiro, obrigatório)
- ativo (booleano, predefinição: true)
- data_criacao (data/hora)

SITUAÇÃO 1

Adicionar um novo produto (CREATE)

Enunciado

O Ricardo recebeu um novo modelo de portátil do fornecedor e precisa de adicioná-lo ao catálogo online. O produto tem as seguintes características:

- Nome: "Portátil ASUS VivoBook 15"
- Categoria: "Portáteis"
- Preço: 649.99€
- Stock inicial: 25 unidades

Query em SQL

```
INSERT INTO produtos (nome, categoria, preco, stock, ativo, data_criacao)
VALUES (
  'Portátil ASUS VivoBook 15',
  'Portáteis',
  649.99,
  25,
  true,
  CURRENT_TIMESTAMP
);
```

Raciocínio por trás da decisão

1. **Campos especificados:** Incluímos todos os campos relevantes exceto id, que é gerado automaticamente pelo sistema.
2. **Valores corretos:**
 - Nome e categoria como texto entre aspas
 - Preço como valor decimal (não texto)
 - Stock como número inteiro
 - ativo definido como true porque o produto está imediatamente disponível
 - data_criacao utiliza CURRENT_TIMESTAMP para registar automaticamente o momento da inserção
3. **Validações implícitas:** O sistema verificará automaticamente se os campos obrigatórios estão preenchidos e se os tipos de dados correspondem ao schema definido.

Resultado

O sistema cria um novo registo na tabela produtos com um id único (por exemplo, 158) e o produto fica imediatamente disponível para consulta e venda no website.

Principais aprendizagens

- CREATE adiciona novos dados ao sistema
- Campos auto-incrementados não precisam de ser especificados
- Funções como CURRENT_TIMESTAMP automatizam valores comuns
- A ordem dos campos deve corresponder à ordem dos valores

SITUAÇÃO 2

Consultar os produtos com stock baixo (READ)

Enunciado

O Ricardo precisa de gerar um relatório dos produtos com stock inferior a 10 unidades para fazer novos pedidos aos fornecedores. Pretende visualizar o nome do produto, categoria, stock atual e preço, ordenados por stock crescente.

Query em SQL

```
SELECT nome, categoria, stock, preco
FROM produtos
WHERE stock < 10 AND ativo = true
ORDER BY stock ASC;
```

Raciocínio por trás da decisão

- 1. **SELECT específico:** Seleccionamos apenas as colunas necessárias para o relatório, evitando a informação desnecessária e melhorando a performance.
- 2. **Filtros aplicados (WHERE):**
 - stock < 10: Apenas produtos com stock baixo
 - ativo = true: Excluimos produtos descontinuados ou inativos
 - Operador AND: Ambas as condições têm de ser verdadeiras
- 3. **Ordenação (ORDER BY):**
 - stock ASC: Ordenação ascendente coloca os produtos com stock mais crítico no topo
 - Facilita a priorização de encomendas

Resultado possível

nome	categoria	stock	preco
-----	-----	-----	-----
Rato Gaming RGB	Periféricos	3	39.99
Teclado Mecânico	Periféricos	5	89.99
Webcam HD 1080p	Periféricos	7	54.99
Monitor 24" Full HD	Monitores	9	179.99

Principais aprendizagens

- READ permite consultar os dados sem os modificar
- WHERE filtra os resultados, conforme critérios específicos
- ORDER BY organiza os resultados para melhor análise
- Operadores lógicos (AND, OR) combinam múltiplas condições

SITUAÇÃO 3

Atualizar o preço do produto (UPDATE)

Enunciado

O fornecedor informou Ricardo de um aumento de preço no modelo "Portátil ASUS VivoBook 15" (id: 158). O novo preço é 679.99€. O Ricardo precisa de atualizar apenas este produto específico.

Query em SQL

```
UPDATE produtos
SET preco = 679.99
WHERE id = 158;
```

Raciocínio por trás da decisão

1. **Identificação precisa:** Utilizamos WHERE id = 158 para garantir que apenas o produto correto é atualizado. O id é único, evitando alterações acidentais noutros produtos.
2. **Campo específico:** Apenas o campo preco é modificado através de SET. Os restantes campos permanecem inalterados.
3. **Segurança:** Se omitíssemos o WHERE, **todos** os produtos teriam o preço alterado para 679.99€ (um erro grave)! O WHERE é essencial.

Verificação recomendada antes da atualização

```
-- Passo 1: Verificar o registo atual
SELECT id, nome, preco FROM produtos WHERE id = 158;

-- Passo 2: Executar a atualização
UPDATE produtos SET preco = 679.99 WHERE id = 158;

-- Passo 3: Confirmar a alteração
SELECT id, nome, preco FROM produtos WHERE id = 158;
```

Resultado

O produto com id 158 tem agora o preço atualizado para 679.99€. Todos os outros campos permanecem inalterados.

Principais aprendizagens

- UPDATE modifica os dados existentes sem criar novos registos
- WHERE é crucial para evitar as alterações não intencionais
- Verificar antes e depois da atualização é uma boa prática
- Podemos atualizar um ou múltiplos campos simultaneamente

SITUAÇÃO 4

Desativar um produto descontinuado (UPDATE com Soft Delete)

Enunciado

A TechStore decidiu descontinuar o modelo "Webcam HD 720p" (id: 89) por ter sido substituído por uma versão melhorada. Em vez de eliminar completamente o registo (pois existem encomendas anteriores que o referenciam), o Ricardo vai marcá-lo como inativo.

Query em SQL

```
UPDATE produtos  
SET ativo = false  
WHERE id = 89;
```

Raciocínio por trás da decisão

- Soft delete vs. Hard delete:** Optamos por marcar o produto como inativo (ativo = false) em vez de eliminá-lo permanentemente. Isto permite:
 - Manter o histórico de vendas consistente
 - Preservar a integridade referencial com tabelas de encomendas
 - Possibilidade de reativar se necessário
 - Cumprir os requisitos de auditoria
- Impacto no sistema:** Queries que consultam produtos para o website devem incluir WHERE ativo = true, assim este produto deixa de aparecer nas listagens públicas.

Query complementar para verificar produtos inativos

```
SELECT id, nome, categoria, stock  
FROM produtos  
WHERE ativo = false  
ORDER BY nome;
```

Resultado

O produto mantém-se na base de dados com todos os seus dados históricos, mas deixa de aparecer no catálogo ativo da loja online.

Principais aprendizagens

- Soft delete preserva os dados importantes, mantendo-os ocultos
- Flags booleanos (true/false) são úteis para estados binários
- A integridade referencial é protegida
- Dados históricos permanecem disponíveis para os relatórios

SITUAÇÃO 5

Atualização em lote após a venda (UPDATE múltiplo)

Enunciado

Um cliente comprou 3 unidades do "Rato Gaming RGB" (id: 142). O Ricardo precisa de atualizar o stock, subtraindo as unidades vendidas. O stock atual é 15 unidades.

Query em SQL

```
UPDATE produtos  
SET stock = stock - 3  
WHERE id = 142;
```

Raciocínio por trás da decisão

1. **Cálculo dinâmico:** Em vez de calcular manualmente ($15 - 3 = 12$) e definir $\text{stock} = 12$, utilizamos $\text{stock} = \text{stock} - 3$. Isto é mais seguro porque:
 - Funciona independentemente do valor atual
 - Reduz os erros humanos no cálculo
 - Permite a reutilização da query
2. **Atomicidade:** Esta operação é atômica. Se duas vendas ocorrerem simultaneamente, o sistema garante que ambas as subtrações são processadas corretamente.

Query de verificação antes da venda

```
SELECT id, nome, stock  
FROM produtos  
WHERE id = 142 AND stock >= 3;
```

Esta verificação garante que existe stock suficiente antes de processar a venda.

Resultado

O stock do produto 142 é reduzido de 15 para 12 unidades.

Principais aprendizagens

- Cálculos relativos ($\text{stock} - 3$) são mais seguros que valores absolutos
- Verificações prévias evitam as vendas de stock inexistente
- Operações aritméticas podem ser feitas diretamente no SQL
- A consistência dos dados é mantida mesmo em operações concorrentes

SITUAÇÃO 6

Eliminar o produto de teste (DELETE)

Enunciado

Durante os testes de sistema, o Ricardo criou um produto fictício "Teste - Não Vender" (id: 999) que nunca foi ativado, nem vendido. Como não tem nenhum valor, nem está referenciado noutras tabelas, pode ser eliminado permanentemente.

Query em SQL

-- Passo 1: Verificar que o produto não tem dependências

```
SELECT * FROM produtos WHERE id = 999;
```

-- Passo 2: Confirmar que não existem encomendas associadas

-- (numa implementação real, verificaríamos as tabelas relacionadas)

-- Passo 3: Eliminar permanentemente

```
DELETE FROM produtos WHERE id = 999;
```

Raciocínio por trás da decisão

- Verificações prévias:** Antes de eliminar, confirmamos:
 - O produto existe
 - Não tem dependências noutras tabelas
 - É realmente um registo de teste sem valor
- DELETE permanente:** Apenas utilizamos DELETE quando temos certeza absoluta que:
 - O registo não tem valor histórico
 - Não existem relações com outros dados
 - A eliminação não viola requisitos legais ou de negócio
- Alternativa preferível:** Na maioria dos casos reais, soft delete (UPDATE ... SET ativo = false) é mais seguro.

Resultado

O registo com id 999 é completamente removido da tabela produtos. Esta ação é irreversível (exceto através de backups).

Principais aprendizagens

- DELETE remove permanentemente os dados da base de dados
- Verificações rigorosas são essenciais antes de eliminar
- DELETE deve ser usado com extrema cautela
- Em produção, soft delete é normalmente preferível
- WHERE é obrigatório para evitar eliminar toda a tabela

Síntese

Ao longo deste exemplo acompanhámos o Ricardo na gestão do catálogo da TechStore, observando como cada operação CRUD serve um propósito específico:

CREATE

Adicionar novos produtos ao catálogo

READ

Consultar as informações para a tomada de decisão (relatórios de stock)

UPDATE

Manter os dados atualizados (preços, stocks, estados)

DELETE

Remover os dados obsoletos (com cuidado)

Fluxo completo numa operação de venda

1. READ → Verificar o stock disponível
2. UPDATE → Reduzir o stock após venda
3. CREATE → Gerar o registo de encomenda (noutra tabela)
4. READ → Confirmar a atualização

Este exemplo demonstra que, na prática empresarial, as operações CRUD trabalham em conjunto para suportar processos de negócio complexos, garantindo que os dados permanecem precisos, atualizados e úteis para a organização.

Glossário

ACID: Conjunto de propriedades que garantem fiabilidade em transações de bases de dados (Atomicity, Consistency, Isolation, Durability).

Constraint: Regra aplicada a uma coluna ou tabela para garantir integridade dos dados (exemplo: NOT NULL, UNIQUE).

Integridade referencial: Garantia de que relações entre tabelas permanecem consistentes (chaves estrangeiras válidas).

Schema: Estrutura que define a organização dos dados numa base de dados (tabelas, colunas, tipos de dados, relações).

Soft Delete: Técnica de marcar registos como inativos em vez de eliminá-los fisicamente da base de dados.

SQL Injection: Vulnerabilidade de segurança onde código malicioso é inserido em queries SQL.

Transação: Conjunto de operações executadas como uma única unidade lógica de trabalho.

Questões frequentes

1. Por que razão não devemos eliminar dados permanentemente em muitos casos?

Existem várias razões para preferir soft delete (marcar como inativo) em vez de eliminação permanente:

- Requisitos legais de retenção de dados (contabilidade, RGPD)
- Necessidade de auditoria e histórico
- Possibilidade de recuperação em caso de erro
- Manutenção de integridade referencial

2. Qual é a operação CRUD mais comum em sistemas empresariais?

A operação READ é de longe a mais frequente, representando 70-90% das interações. Isto acontece porque os dados são consultados múltiplas vezes ao longo do seu ciclo de vida (relatórios, dashboards, pesquisas, validações, etc.)

3. Como escolher entre SQL e NoSQL para as minhas operações CRUD?

A escolha depende de vários fatores:

- **SQL:** Dados estruturados, relações complexas, necessidade de transações ACID, consistência forte
- **NoSQL:** Dados não estruturados, necessidade de escalabilidade horizontal, flexibilidade de schema, performance em leituras

4. O que são operações CRUD em lote (batch)?

São operações que afetam múltiplos registos simultaneamente. Por exemplo, atualizar preços de todos os produtos de uma categoria ou eliminar todos os registos anteriores a determinada data. Requerem atenção especial devido ao impacto abrangente.

5. Como garantir a segurança nas operações CRUD?

Principais medidas:

- Validação rigorosa de inputs
- Uso de prepared statements (SQL) para evitar a injeção
- Controlo de acesso baseado em papéis (RBAC)
- Registo de ações sensíveis (audit logs)
- Encriptação de dados sensíveis